

CSA0624 - Design and Analysis of Algorithms

CO2_AT3 Debugging & Optimisation Task

Name: Aryan Arun

Reg.No: 192411157

Q1. Debugging Selection Sort Code

Original (Buggy) Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]:    # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

1. Identifying the Errors

Error 1 — Wrong comparison operator (critical logic error): The inner loop is meant to locate the smallest element in the unsorted segment so it can be moved to the front, but the condition `arr[j] > arr[min_idx]` instead keeps track of the largest element seen so far. This is a direct inversion of Selection Sort's core logic.

Root cause: Selection Sort must repeatedly find the minimum of the remaining unsorted sub-array. Using `'>'` makes the algorithm select the maximum at every pass instead, so the array ends up sorted in descending order rather than ascending order — for example, `[64, 25, 12, 22, 11]` would incorrectly become `[64, 25, 22, 12, 11]` instead of `[11, 12, 22, 25, 64]`.

Error 2 — Unnecessary final pass (minor inefficiency, not a correctness bug): The outer loop runs for `i` in `range(n)`, including `i = n-1`. At this point the inner loop `range(n, n)` does not execute, so `min_idx` stays equal to `i`, and the swap `arr[i], arr[i] = arr[i], arr[i]` is a redundant self-swap. It does not break correctness but wastes a swap operation on every call.

2. Step-by-Step Debugging

- Step 1 — Trace the intent: Selection Sort should place the smallest unfound element at position `i` on each pass.
- Step 2 — Run the buggy condition mentally on `[64, 25, 12, 22, 11]`: Pass 1 compares elements and keeps updating `min_idx` whenever a larger value appears, ending with `min_idx` pointing at 64 (already at index 0) — so nothing swaps, and larger values keep 'winning', confirming the algorithm is selecting maximums, not minimums.
- Step 3 — Correct the comparison: change `arr[j] > arr[min_idx]` to `arr[j] < arr[min_idx]`, so `min_idx` updates only when a smaller element is found.
- Step 4 — Remove the redundant last pass: change `range(n)` to `range(n-1)` in the outer loop, since after placing the smallest `n-1` elements correctly, the last element is automatically in its correct position.
- Step 5 — Guard the swap: only perform the swap if `min_idx != i`, avoiding a wasted no-op swap when the current element is already the minimum.

3. Optimizing the Implementation

- Reduce the outer loop from `n` iterations to `n-1`, since the last element never needs its own pass.
- Skip the swap operation entirely when `min_idx == i` (the current element is already correctly placed), avoiding an unnecessary write to the array.
- These optimizations reduce the number of swaps performed but do not change the number of comparisons, which remains inherent to Selection Sort's design.

4. Time Complexity of the Corrected Algorithm

Case	Comparisons	Swaps	Complexity
Best Case	$n(n-1)/2$ (always, regardless of order)	0 (if already sorted)	$O(n^2)$
Average Case	$n(n-1)/2$	Close to $n-1$	$O(n^2)$
Worst Case	$n(n-1)/2$	$n-1$	$O(n^2)$

Comparisons remain $O(n^2)$ in every case since the algorithm must always scan the remaining unsorted segment to confirm the minimum; the optimization only reduces the constant-factor overhead of swaps, not the asymptotic complexity. Space complexity remains $O(1)$, as sorting is done in place.

5. Final Corrected and Optimized Code

```
def selection_sort(arr):
    """
    Sorts a list in ascending order using the Selection Sort algorithm.

    Fixes applied:
    1. Comparison operator corrected from '>' to '<' so the algorithm
       tracks the MINIMUM element of the unsorted segment (not the maximum).
    2. Outer loop reduced to range(n-1): the last element is guaranteed
       to be in place once the first n-1 elements are sorted.
    3. Swap is skipped when the current element is already the minimum,
       avoiding a redundant no-op swap.
    """
    n = len(arr)
    for i in range(n - 1):
        # last element needs no pass
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                # FIXED: find minimum, not maximum
                min_idx = j
        if min_idx != i:
            # avoid unnecessary self-swap
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Verified output: `selection_sort([64, 25, 12, 22, 11])` → `[11, 12, 22, 25, 64]` (correctly ascending).

Q2. Debugging Linear Search Code

Original (Buggy) Code

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return -1    # ERROR  
    return i            # ERROR
```

1. Identifying the Errors

Error 1 — Inverted return on match: When a match is found (`arr[i] == key`), the function returns `-1`, which is the conventional signal for 'not found'. This means a successful search is reported as a failure.

Error 2 — Wrong return after the loop completes: If no match is found, the loop finishes naturally and the function returns `i`. At that point, `i` simply holds the last index examined (`len(arr) - 1`), not a 'not found' indicator — so a failed search returns a seemingly valid index instead of `-1`.

Error 3 — Hidden edge-case crash: If `arr` is empty, the `for` loop body never executes, so the variable `i` is never assigned. The final `return i` then raises an `UnboundLocalError` instead of gracefully returning 'not found'.

Root cause: The two `return` statements are effectively swapped from their intended roles — success and failure signals are reversed, and the failure path incorrectly relies on a loop variable rather than an explicit sentinel value.

2. Debugging the Code

- Swap the two `return` statements: return the matching index `i` when `arr[i] == key` is true, and return `-1` only after the loop has fully completed without finding a match.
- This also automatically fixes the empty-list crash, since the corrected 'return `-1`' is outside and after the loop, and does not depend on `i` being defined.

3. Optimizing for Readability and Efficiency

- Use Python's `enumerate()` to iterate over (index, value) pairs directly, avoiding repeated indexing (`arr[i]`) inside the loop — slightly more efficient and more idiomatic/readable.
- Add a docstring and type-hint-style comments so the function's contract (return index or `-1`) is unambiguous to future maintainers.

4. Time Complexity Analysis

Case	Scenario	Complexity
Best Case	Key is the first element	$O(1)$
Average Case	Key is found somewhere in the middle, on average	$O(n)$
Worst Case	Key is the last element, or absent entirely	$O(n)$

This is justified because Linear Search has no information about the arrangement of elements and must examine them one at a time until a match is found or the array is exhausted; in the worst case, all n elements are examined.

5. Final Corrected and Optimized Code

```
def linear_search(arr, key):  
    """  
    Searches for 'key' in 'arr' using sequential scanning.  
  
    Fixes applied:  
    1. Returns the matching index i (not -1) as soon as a match is found.  
    2. Returns -1 only after the full loop completes without a match,  
       instead of relying on the leftover loop variable i.  
    3. Using enumerate() also removes any dependency on a loop variable  
       existing after the loop, so an empty list safely returns -1.  
  
    Returns:  
    The index of the first occurrence of 'key', or -1 if not found.  
    """  
    for index, value in enumerate(arr):  
        if value == key:  
            return index      # FIXED: return index on success  
    return -1                 # FIXED: return -1 only if no match was found
```

Verified: `linear_search([5,3,8,1], 8) → 2`, `linear_search([5,3,8,1], 100) → -1`, `linear_search([], 5) → -1` (no crash).

Q3. Debugging Closest Pair Code

Original (Buggy) Code

```
import math

def closest_pair(points):
    min_dist = 0    # ERROR
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                              (points[i][1]-points[j][1])**2)
            if dist > min_dist:    # ERROR
                min_dist = dist
    return min_dist
```

1. Identifying the Errors

Error 1 — Wrong initial value: `min_dist` is initialized to 0, but 0 is the smallest possible distance, not a safe 'no distance seen yet' placeholder for a minimum-tracking variable. It should start at positive infinity (or the first computed distance) so that any real distance can replace it.

Error 2 — Wrong comparison direction: The condition `dist > min_dist` keeps replacing `min_dist` with distances that are LARGER than the current value. Combined with the 0 initial value, this makes the function converge on the MAXIMUM pairwise distance, not the minimum — the algorithm as written finds the farthest pair, which is the opposite of its stated purpose.

Error 3 — Missing the actual closest pair: Even conceptually, the function only returns a distance value, never the pair of points that produced it, making the result far less useful for any application that needs to know which two points are closest.

2. Debugging Step-by-Step

- Step 1 — Change `min_dist = 0` to `min_dist = math.inf`, so the very first computed distance always becomes the initial candidate minimum.
- Step 2 — Change `dist > min_dist` to `dist < min_dist`, so the variable is updated only when a smaller distance is found.
- Step 3 — Track and return the actual pair of points alongside the distance, not just the numeric value.
- Step 4 — Add a guard clause: if fewer than 2 points are given, raise a clear error rather than returning an undefined/infinite result.

3. Optimizing to Avoid Redundant Calculations

Since the square root function is monotonically increasing, comparing squared distances ($dx^2 + dy^2$) gives exactly the same ordering as comparing actual distances — without needing a `sqrt()` call inside the $O(n^2)$ loop. The expensive `sqrt()` is applied only once, to the final minimum, at the very end. This removes up to $n(n-1)/2$ unnecessary `sqrt()` calls, leaving only one.

4. Time Complexity of the Corrected Version

The number of point pairs examined is still $C(n,2) = n(n-1)/2$, so the asymptotic time complexity remains $O(n^2)$. The optimization does not change this asymptotic bound — it reduces the constant factor per iteration by replacing an expensive `sqrt()` operation with cheaper integer/float multiplication, which matters in practice for large point sets even though the Big-O class is unchanged.

5. Final Corrected and Optimized Code

```
import math

def closest_pair(points):
    """
    Brute-force closest pair of points.

    Fixes applied:
    1. min_dist starts at infinity (not 0), so any real distance
       can correctly become the first candidate minimum.
    2. Comparison changed from '>' to '<' so the function tracks
       the MINIMUM distance, not the maximum.
    3. The actual closest pair of points is tracked and returned,
       not just the distance value.

    Optimization:
    - Squared distances are compared during the loop (avoiding a
      sqrt() call at every iteration); sqrt() is applied only once,
      to the final minimum, since sqrt() preserves ordering.

    Returns:
    (min_distance, (point_a, point_b))
    """
    n = len(points)
    if n < 2:
        raise ValueError("At least two points are required.")

    min_sq_dist = math.inf      # FIXED: start at infinity, not 0
    closest_points = None

    for i in range(n):
        for j in range(i + 1, n):
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            sq_dist = dx * dx + dy * dy      # avoid sqrt inside the loop
            if sq_dist < min_sq_dist:        # FIXED: track minimum
                min_sq_dist = sq_dist
                closest_points = (points[i], points[j])

    return math.sqrt(min_sq_dist), closest_points  # sqrt computed once
```

Verified: `closest_pair([(2,3),(12,30),(40,50),(5,1),(12,10),(3,4)])` → `(1.4142..., ((2,3),(3,4)))` — matching the expected closest pair.

Q4. Debugging Convex Hull Logic

Given Code Fragment

```
def orientation(p, q, r):  
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])    # SIGN ISSUE  
# used incorrectly in hull condition
```

1. Identifying the Logical Errors

Understanding orientation() itself: The formula computed is the standard cross-product-based orientation test: a positive result means point r is to one side (counter-clockwise turn) of the line through p and q , a negative result means the other side (clockwise turn), and 0 means the three points are collinear. The formula itself is correct — the bug lies in how its sign is used to decide hull membership.

Error — Only one-sided validity check: A brute-force hull edge test must confirm that ALL other points lie on the SAME side of the candidate line — either every remaining point gives a non-negative orientation, OR every remaining point gives a non-positive orientation. A common but incorrect implementation checks only one of these two conditions (e.g., rejecting an edge the moment any orientation is negative), without also accepting the symmetric case where all points are consistently negative. This one-sided check silently discards true hull edges that happen to be traversed in the opposite rotational direction, while also failing to properly disqualify an interior segment that has points scattered on both sides.

Impact: Because true hull edges can be missed while some interior chords are not correctly disqualified, the algorithm can end up including interior points in the returned boundary — exactly the incorrect behaviour described in the scenario.

2. Debugging the Algorithm

- Step 1 — For each candidate edge ($\text{points}[i], \text{points}[j]$), compute the orientation of every other point k relative to that line.
- Step 2 — Track whether any positive orientation value was seen ($\text{pos} = \text{True}$) and whether any negative value was seen ($\text{neg} = \text{True}$), allowing exactly 0 (collinear) to count toward either side without disqualifying the edge.
- Step 3 — The edge is a valid hull edge only if points are NOT found on both sides — i.e., NOT (pos and neg). This correctly accepts edges where all points are consistently on one side (or exactly on the line), and correctly rejects edges where points are found on both sides (a true interior chord).
- Step 4 — Add an early-exit break the moment both pos and neg become true, since the edge is already disqualified and further points don't need to be checked.

3. Optimizing to Reduce Unnecessary Checks

- Iterate only over unordered pairs (i, j) with $j > i$, instead of all ordered pairs — this halves the number of candidate edges examined, from $n(n-1)$ down to $n(n-1)/2$, since edge (i, j) and (j, i) represent the same line segment.
- Break out of the inner point-checking loop as soon as points are confirmed on both sides (pos and neg both true) — there is no need to keep checking the remaining points once an edge is already disqualified.

4. Time Complexity of the Corrected Solution

For n points, there are $O(n^2)$ candidate edges (after removing duplicate ordered pairs), and in the worst case each is checked against $O(n)$ other points, giving an overall worst-case time complexity of $O(n^3)$. The early-break optimization does not change this worst-case bound (a genuine hull edge still requires checking every other

point to confirm consistency), but it substantially reduces the average-case running time, since most interior candidate segments are disqualified after only a few checks rather than scanning all n points.

5. Final Corrected and Optimized Code

```
def orientation(p, q, r):
    """
    Orientation of ordered triplet (p, q, r):
    > 0 -> counter-clockwise turn (r is left of line p->q)
    < 0 -> clockwise turn (r is right of line p->q)
    = 0 -> collinear
    """
    return (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])

def convex_hull_brute_force(points):
    """
    Brute-force convex hull.

    Fix applied:
    A segment (points[i], points[j]) is a valid hull edge only if every
    other point lies on the SAME side of the line through it (or exactly
    on it). The previous one-sided check allowed true hull edges to be
    missed and interior chords to be wrongly retained.

    Optimizations:
    - Only unordered pairs (j > i) are tested, halving redundant edges.
    - The inner loop breaks early as soon as points are found on both
      sides, since the edge is already disqualified.
    """
    n = len(points)
    hull_edges = []

    for i in range(n):
        for j in range(i + 1, n):          # unordered pairs only
            pos = neg = False
            for k in range(n):
                if k == i or k == j:
                    continue
                o = orientation(points[i], points[j], points[k])
                if o > 0:
                    pos = True
                elif o < 0:
                    neg = True
                if pos and neg:             # points on both sides -> disqualified
                    break
            if not (pos and neg):           # all points on one side (or collinear)
                hull_edges.append((points[i], points[j]))

    return hull_edges
```

Verified on $\{(0,3),(2,2),(1,1),(2,1),(3,0),(0,0),(3,3)\}$: the corrected function returns exactly the four true boundary edges — $(0,3)-(0,0)$, $(0,3)-(3,3)$, $(3,0)-(0,0)$, $(3,0)-(3,3)$ — correctly excluding the interior points $(1,1)$, $(2,1)$, $(2,2)$.

Q5. Debugging Binary Search Code

Original (Buggy) Code

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            high = mid - 1    # ERROR
        else:
            low = mid + 1     # ERROR
    return -1
```

1. Identifying the Errors

Error 1 — Boundary updates are swapped: When $\text{arr}[\text{mid}] < \text{key}$, the target must lie in the RIGHT half of the current range, so low should move to $\text{mid} + 1$ — but the code instead updates $\text{high} = \text{mid} - 1$, incorrectly narrowing to the left half. Symmetrically, when $\text{arr}[\text{mid}] > \text{key}$ (the else branch), the target must lie in the LEFT half, so high should move to $\text{mid} - 1$ — but the code updates $\text{low} = \text{mid} + 1$, incorrectly narrowing to the right half. The two updates are completely reversed, so the search actively discards the half of the array that actually contains the key.

Error 2 — Loop condition excludes the final candidate: The condition $\text{while low} < \text{high}$ stops as soon as $\text{low} == \text{high}$, but at that point there is still one valid element left to check ($\text{arr}[\text{low}]$ itself). Using strict ' $<$ ' means this last remaining single-element range is never tested, so the algorithm can incorrectly report 'not found' (-1) even when the key is present as the sole remaining candidate.

Error 3 — Midpoint calculation, while functionally fine in Python, is not overflow-safe: $(\text{low} + \text{high}) // 2$ can overflow the maximum integer size in fixed-width-integer languages (C, Java) when low and high are both very large, since their sum can exceed the integer limit before the division happens. Python integers don't overflow, but for portability and best practice the safer form $\text{low} + (\text{high} - \text{low}) // 2$ should be used, which never needs a sum larger than high itself.

2. Debugging the Code

- Fix boundary updates: when $\text{arr}[\text{mid}] < \text{key}$, set $\text{low} = \text{mid} + 1$ (search right half); when $\text{arr}[\text{mid}] > \text{key}$, set $\text{high} = \text{mid} - 1$ (search left half).
- Fix the loop condition: change $\text{while low} < \text{high}$ to $\text{while low} \leq \text{high}$, so the final single-element range ($\text{low} == \text{high}$) is still correctly examined before giving up.
- Use the overflow-safe midpoint formula: $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$.

3. Optimizing to Avoid Overflow

Replacing $(\text{low} + \text{high}) // 2$ with $\text{low} + (\text{high} - \text{low}) // 2$ produces the exact same midpoint value arithmetically, but avoids ever computing a sum that could exceed the maximum representable integer in languages with fixed-size integer types. This is a standard, well-known defensive optimization for Binary Search implementations intended to be robust and portable.

4. Performance Analysis

Case	Scenario	Complexity
Best Case	The key is found at the very first midpoint checked	$O(1)$
Average Case	The search space is halved repeatedly until a match is found	$O(\log n)$

Case	Scenario	Complexity
Worst Case	The key is absent, or is found only after narrowing down to a single element	$O(\log n)$

This is justified because each iteration of Binary Search eliminates half of the remaining candidates, so after k iterations at most $n/2^k$ elements remain; the search terminates once this shrinks to a single element, which takes $k = \lceil \log_2 n \rceil$ iterations — giving the $O(\log n)$ bound for both average and worst cases.

5. Final Corrected and Optimized Code

```
def binary_search(arr, key):
    """
    Iterative Binary Search on a sorted array.

    Fixes applied:
    1. Loop condition changed from 'low < high' to 'low <= high' so the
       final single-element range (low == high) is still checked.
    2. Boundary updates corrected: arr[mid] < key now moves 'low' up
       (search right half); arr[mid] > key now moves 'high' down
       (search left half) — previously these were swapped.
    3. Midpoint calculated as low + (high - low) // 2 instead of
       (low + high) // 2, avoiding integer overflow in languages with
       fixed-size integers.

    Returns:
    The index of 'key' in the sorted array 'arr', or -1 if not found.
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Verified on [11,12,22,25,45,64,90]: $\text{binary_search}(\text{arr}, 22) \rightarrow 2$, $\text{binary_search}(\text{arr}, 11) \rightarrow 0$ (first element), $\text{binary_search}(\text{arr}, 90) \rightarrow 6$ (last element), $\text{binary_search}(\text{arr}, 13) \rightarrow -1$ (not found), and single-element cases $\text{binary_search}([5], 5) \rightarrow 0$ and $\text{binary_search}([5], 3) \rightarrow -1$ all resolve correctly — confirming the boundary-condition fix works.

Summary Table

Q	Algorithm	Core Bug	Complexity After Fix
1	Selection Sort	Wrong comparison operator ('>' instead of '<') selects maximum, not minimum	$O(n^2)$
2	Linear Search	Success/failure return statements swapped	$O(n)$
3	Closest Pair	min_dist initialized to 0 and compared with '>', finding maximum not minimum	$O(n^2)$
4	Convex Hull	One-sided orientation check lets interior chords pass as hull edges	$O(n^3)$
5	Binary Search	Boundary updates swapped; loop excludes final single-element range	$O(\log n)$